1. Display the processes associated with the current user using ps and observe the PID, terminal and process status.

```
akansha@localhost:~$ ps
    PID TTY          TIME CMD
   3241 pts/0    00:00:00 bash
   4215 pts/0    00:00:00 ps
```

2. Display all processes running on the system using an appropriate ps option and identify the PID, user and command fields.

```
akansha@localhost:~$ ps -ef | head -n 10
UID          PID    PPID  C STIME TTY          TIME CMD
root           1       0  0 08:30 ?        00:00:02 /usr/lib/systemd/systemd --
switched-root --system --deserialize 30
root           2       0  0 08:30 ?        00:00:00 [kthreadd]
root           3       2  0 08:30 ?        00:00:00 [rcu_gp]
root           4       2  0 08:30 ?        00:00:00 [rcu_par_gp]
akansha     3241    3240  0 10:15 pts/0    00:00:00 bash
akansha     4218    3241  0 11:45 pts/0    00:00:00 ps -ef
```

3. Display a detailed process listing using ps aux and identify the CPU and memory usage fields.

```
akansha@localhost:~$ ps aux | head -n 6
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.4 178540 13420 ?        Ss   08:30   0:02 /usr/lib/
systemd/systemd
akansha     3241  0.0  0.2 225412  5120 pts/0    Ss   10:15   0:00 bash
akansha     4222  0.0  0.1 222100  2400 pts/0    R+   11:46   0:00 ps aux

Identification:
- %CPU: Percentage of CPU time used by the process.
- %MEM: Percentage of physical memory (RAM) used by the process.
```

4. Display the process hierarchy using ps with an option that shows parent-child relationships.

```
akansha@localhost:~$ ps -ef --forest | head -n 12
UID          PID    PPID  C STIME TTY          TIME CMD
root           1       0  0 08:30 ?        00:00:02 /usr/lib/systemd/systemd
root         850       1  0 08:30 ?        00:00:00  \_ /usr/sbin/sshd -D
root        3238     850  0 10:15 ?        00:00:00      \_ sshd: akansha [priv]
akansha     3240    3238  0 10:15 ?        00:00:00          \_ sshd:
akansha@pts/0
akansha     3241    3240  0 10:15 pts/0    00:00:00              \_ -bash
akansha     4225    3241  0 11:48 pts/0    00:00:00                  \_ ps -ef
--forest
```

5. Start the top command and observe the running processes, CPU usage, memory usage and process IDs.

```
akansha@localhost:~$ top

top - 11:50:00 up 3:20,  1 user,  load average: 0.05, 0.03, 0.00
Tasks: 185 total,   1 running, 184 sleeping,   0 stopped,   0 zombie
%Cpu(s):  1.5 us,  0.5 sy,  0.0 ni, 98.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   3850.4 total,   1420.2 free,   1210.5 used,   1219.7 buff/cache
MiB Swap:   2048.0 total,   2048.0 free,      0.0 used.   2410.1 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
 4230 akansha   20   0  235400   4820   3120 R   0.7   0.1   0:00.03 top
    1 root      20   0  178540  13420   9200 S   0.0   0.4   0:02.15 systemd
```

6. Use top to identify a process with high CPU or memory usage and record its PID.

```
# Inside top interactive interface:
# Press 'P' to sort processes by CPU usage
# Press 'M' to sort processes by Memory usage
# Press 'q' to quit top

Recorded High Resource Process PID: 4230 (Command: top / bash)
```

7. Start htop and use its interactive interface to view processes and resource usage.

```
akansha@localhost:~$ htop

# Interactive htop controls:
# F3: Search process
# F4: Filter processes by name
# F5: Tree view (shows parent-child relationship)
# F6: Sort by CPU%, MEM%, PID, USER
# F10 or 'q': Quit htop interface
```

8. Start a long-running command in the foreground and terminate it using Ctrl+C.

```
akansha@localhost:~$ sleep 300
^C
akansha@localhost:~$ echo $?
130
```

9. Start a long-running command in the background using the & operator and display the assigned job information.

```
akansha@localhost:~$ sleep 500 &
[1] 4312
akansha@localhost:~$ echo "Background job started with Job ID [1] and PID 4312"
Background job started with Job ID [1] and PID 4312
```

10. Display the jobs currently running in the shell using the jobs command.

```
akansha@localhost:~$ jobs
[1]+  Running                 sleep 500 &
```

11. Bring a background job to the foreground using the fg command and then terminate it safely.

```
akansha@localhost:~$ fg %1
sleep 500
^C
akansha@localhost:~$ jobs
akansha@localhost:~$
```

12. Suspend a running foreground process using Ctrl+Z and verify that the job has been stopped.

```
akansha@localhost:~$ sleep 600
^Z
[1]+  Stopped                 sleep 600
akansha@localhost:~$ jobs
[1]+  Stopped                 sleep 600
```

13. Resume a stopped job in the background using bg and verify its status using jobs.

```
akansha@localhost:~$ bg %1
[1]+ sleep 600 &
akansha@localhost:~$ jobs
[1]+  Running                 sleep 600 &
```

14. Use jobs -l to display the job number and PID of background or stopped jobs.

```
akansha@localhost:~$ jobs -l
[1]+  4350 Running                 sleep 600 &
```

15. Use kill with a selected PID to terminate a process and verify that the process has ended.

```
akansha@localhost:~$ kill 4350
[1]+  Terminated              sleep 600
akansha@localhost:~$ jobs
akansha@localhost:~$
```

16. Use kill -TERM with a selected PID and explain why SIGTERM is normally preferred before forceful termination.

```
akansha@localhost:~$ sleep 400 &
[1] 4402
akansha@localhost:~$ kill -TERM 4402
[1]+  Terminated              sleep 400
```

Explanation:
SIGTERM (Signal 15) allows the process to clean up temporary files, release locks, and terminate gracefully, preventing data corruption or resource leaks unlike SIGKILL.

17. Use kill -KILL with a selected test process when normal termination does not work, and verify the result.

```
akansha@localhost:~$ sleep 400 &
[1] 4415
akansha@localhost:~$ kill -KILL 4415
[1]+  Killed                  sleep 400
akansha@localhost:~$ jobs
akansha@localhost:~$
```

18. Use pkill to terminate a selected test process by its process name and verify that it has ended.

```
akansha@localhost:~$ sleep 700 &
[1] 4430
akansha@localhost:~$ pkill sleep
[1]+  Terminated              sleep 700
akansha@localhost:~$ pgrep sleep
akansha@localhost:~$
```

19. Use ps, jobs and appropriate process-control commands to identify and manage multiple test processes safely.

```
akansha@localhost:~$ sleep 300 &
[1] 4450
akansha@localhost:~$ sleep 400 &
[2] 4451
akansha@localhost:~$ jobs -l
[1]-  4450 Running                 sleep 300 &
[2]+  4451 Running                 sleep 400 &
akansha@localhost:~$ kill 4450 4451
[1]-  Terminated              sleep 300
[2]+  Terminated              sleep 400
```

20. Perform a complete process-management workflow: inspect processes, start a foreground job, move a job to the background, suspend/resume it, terminate it with kill, and clean up all test processes.

```
akansha@localhost:~$ ps aux | grep bash
akansha     3241  0.0  0.2 225412  5120 pts/0    Ss   10:15   0:00 bash
akansha@localhost:~$ sleep 800
^Z
[1]+  Stopped                 sleep 800
akansha@localhost:~$ bg %1
[1]+ sleep 800 &
akansha@localhost:~$ jobs -l
[1]+  4480 Running                 sleep 800 &
akansha@localhost:~$ kill -15 4480
[1]+  Terminated              sleep 800
akansha@localhost:~$ pgrep sleep
akansha@localhost:~$
```